TECHNICAL WALKTHROUGH · BACKEND ENGINEERING CHALLENGE

Vantedge Bid

Tender document ingestion and compliance auditing, so a bid is not disqualified for a clerical reason.

NestJS 11 · TypeScript | PostgreSQL 16 | Redis + BullMQ | S3-compatible / local-disk storage

Main walkthrough: slides 2–17 · Technical appendix A1–A7: slides 19–25

01 · THE PROBLEM

Bids fail on paperwork, not on price

Tender bids are routinely disqualified for clerical reasons. The checks are easy but tedious, and done by hand under deadline pressure.

SBU CERTIFICATE

Expired at the deadline

Valid today, expired on the day bids are opened.

```
rule  SBU.NOT_EXPIRED
```

CLASSIFICATION

Codes don't match

Certificate subclassification differs from what the announcement requires.

```
rule  SBU.CLASSIFICATION_MATCH
```

BID BOND

Amount too low

Bond below the required percentage of the estimated package value.

```
rule  BOND.AMOUNT_SUFFICIENT
```

Ingestion turns documents into facts. **Compliance auditing** turns facts into an explainable verdict.

What was asked, what was built, what was left out

Asked for

as the README states the brief

- Tender Documents module: upload, validate, process, extract
- Compliance Audit module against a versioned ruleset
- PostgreSQL schema + ERD
- Ingestion REST API, queue and worker
- Request idempotency and de-duplication
- OpenAPI documentation

Implemented and tested

present in src/, covered by tests

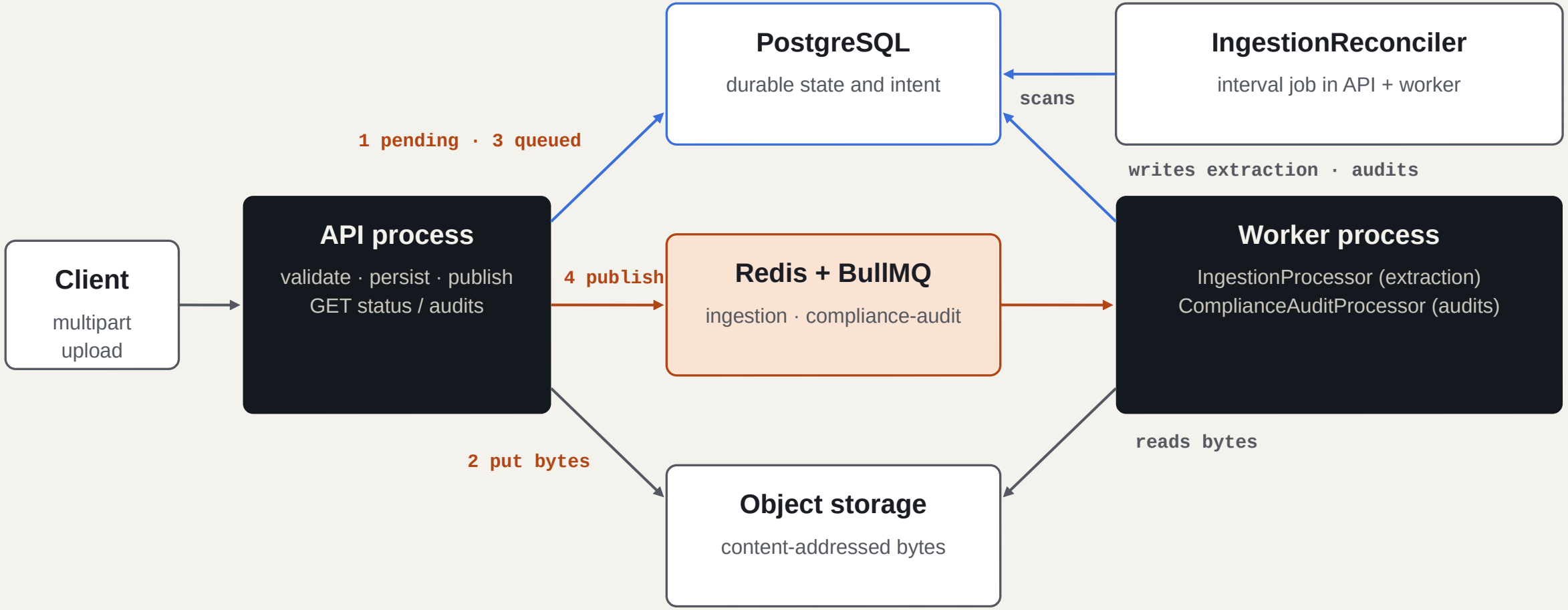
- Tender registration opens one slot per mandatory document type
- Upload gate with eight checks, then 202 Accepted
- Slot / version model, content-addressed storage
- Idempotent uploads with crash recovery
- Separate worker: extraction, then debounced audits
- Rule-based audits with per-finding evidence
- 6 SQL migrations · 5 ADRs · Swagger · Compose · CI

Deliberately not built

documented as gaps, not hidden

- Authentication / authorisation (header is trusted)
- Row-level security, composite tenant keys
- Malware scanning, OCR
- Expiry / deadline re-audit scheduler
- Pagination and rate limiting on the main API
- Presigned uploads (method exists, no endpoint)
- AI-based compliance decisions

Two processes, one durable record



Commit intent to PostgreSQL first, then publish. A worker only ever sees durable state. Redis holds schedules, not truth; the reconciler re-drives work from Postgres.

From tender registration to an explainable verdict

1 Register tender — INGESTION

COMPONENT	POST /v1/tenders · TendersService.create
TABLES	`tenders · tender_documents`
OUTPUT	Tender + empty slot per mandatory type

2 Upload + validate — INGESTION

COMPONENT	POST …/documents · FileValidatorService
TABLES	`idempotency_keys`
OUTPUT	8 checks pass, SHA-256 computed

3 Store + queue — INGESTION

COMPONENT	DocumentsService.persist · StorageService
TABLES	`document_versions · ingestion_jobs`
OUTPUT	Version queued · 202 + statusUrl

4 Extract — INGESTION

COMPONENT	Worker: IngestionProcessor · ExtractionService
TABLES	`document_extractions · audit_trail`
OUTPUT	processed (or failed / quarantined)

5 Audit — COMPLIANCE

COMPONENT	ComplianceQueueService · ComplianceAuditProcessor
TABLES	`tender_audit_state · compliance_audits`
OUTPUT	Debounced run on its own queue

6 Findings + verdict — COMPLIANCE

COMPONENT	ComplianceService · EvaluatorRegistry
TABLES	`compliance_findings`
OUTPUT	Verdict, score, evidence per finding

When does an audit run? Automatically only after a version reaches processed (failed or quarantined versions request none), debounced per tender. Or on demand: POST /v1/tenders/:id/compliance-audits runs one synchronously.

A slot is the requirement, a version is a file

`tenders`

Tender

A bid in preparation: code, buyer, deadline, required SBU codes, estimated value.

→

`tender_documents`

Slot

"This bid needs an SBU." Points at its current version, which only moves forward.

→

`document_versions`

Version

One uploaded file. A correction is a new version; nothing is overwritten.

→

`document_extractions`

Extraction

Fields read from one version (UNIQUE version_id).

LIFECYCLE `pending` → `queued` → `processing` → `processed` or `failed` `quarantined`

Hybrid extraction storage (ADR 0004)

TYPED COLUMNS · rules, filters and joins depend on them

`document_number · holder_name · holder_npwp · issued_at · expires_at · classification_codes · monetary_value`

extracted_data JSONB · type-specific extras

`kbliCodes · qualificationGrade · guarantor · competencyLevel`

Promotion rule: a field earns a column once something queries it.

15 tables, grouped by responsibility

TENANCY

organizations tenant root

users members; uploader reference

TENDER MANAGEMENT

tenders a bid being prepared

document_types reference data, limits

DOCUMENTS

tender_documents slot (requirement)

document_versions uploaded file

document_extractions fields read

PROCESSING + RELIABILITY

ingestion_jobs job ledger / outbox

idempotency_keys request claims

COMPLIANCE

compliance_rulesets versioned regulation

compliance_rules rule rows

compliance_audits one run

compliance_findings per-rule results

COORDINATION + HISTORY

tender_audit_state an audit is owed

audit_trail who changed what

Enforced by PostgreSQL 26 foreign keys · unique + partial indexes (one active ruleset per code, one extraction per version, checksum per slot) · check constraints

Convention only tenant consistency between parent and child rows · ruleset and audit_trail immutability · no row-level security

The order of the writes is the design

INSIDE THE HTTP REQUEST · API PROCESS

1 Validate
8 checks; SHA-256 computed

→

2 Claim key
Idempotency-Key + request fingerprint

→

3 Record pending
Tx: slot, version pending, claim.version_id

→

4 Store bytes
content-addressed; reused if present

→

5 Promote
Tx: pending → queued, pointer, ledger row

→

6 Publish
ingest-<versionId>, then published_at → 202

ASYNCHRONOUS · WORKER PROCESS AND CLIENT

7 Process
worker: processing, extract text + fields

→

8 Persist result
upsert extraction; processed / failed / quarantined

→

9 Poll status
GET /v1/documents/versions/:id/status

Audit is requested after step 8 only if the version is processed.

Database before queue

Intent is committed before the job is published. A worker can only see durable state; Redis schedules, Postgres decides.

Idempotent upload

Same key + same fingerprint replays the stored response or recovers the original version via claim.version_id. Different fingerprint → 409.

Conditional promotion

WHERE lifecycle = 'pending': only one request wins; a stale one cannot overwrite processed and returns the persisted lifecycle.

Content-addressed storage

Key org/<orgId>/<2 hex>/<sha256><ext>: same bytes stored once per tenant, never client input in the path.

08 · VALIDATION GATE

Eight checks, cheapest first

01
File present
`400 file_missing`

02
Not empty
`400 file_empty`

03
Size limit
`413 file_too_large`

04
MIME allow-list
`415 unsupported_media_type`

05
Magic bytes match
`415 file_signature_mismatch`

06
Extension sanity
`logged, not rejected`

07
PDF not encrypted
`415 file_encrypted`

08
SHA-256 computed
`content address`

Storage keys never contain client input: `org/<orgId>/<2 hex>/<sha256><ext>`. The filename is sanitised and kept only as metadata.

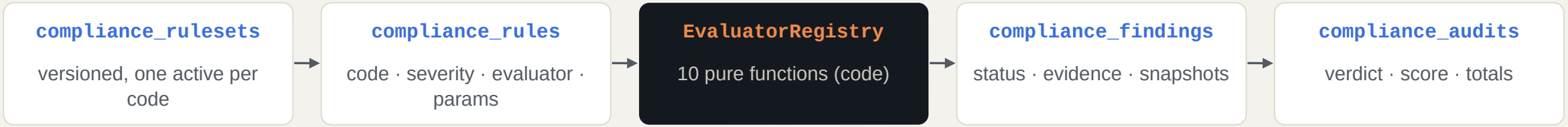
Before the gate: unknown or foreign tender → 404; upload after the submission deadline → 422 tender_deadline_passed.

Every gap between the three stores has a defined recovery

What fails	What is persisted	How it recovers	Mode
API stops mid-upload	`version pending`	Same-key or same-bytes retry	**Client retry**
Commit ok, publish fails	`published_at IS NULL`	Reconciler republishes after grace	**Automatic**
Request dies holding a key	`claim + owner_token + version_id`	Takeover after lease recovers the version	**Client retry**
Extraction fails	`job failed / dead_letter`	Backoff retries; unreadable → quarantined	**Automatic**
Redis loses an audit job	`tender_audit_state.pending`	reconcileAudits reschedules	**Automatic**
Redis loses a published ingestion job	`version queued, published_at set`	Retry of the same upload republishes	**Client retry**
Audit fails	`compliance_audits failed`	Final failure restores pending; document unaffected	**Automatic**
Audit requested mid-run	`pending = true`	finishRun schedules one follow-up	**Automatic**

Delivery is at-least-once; processing is idempotent (deterministic job ids, conditional transitions, extraction upsert). Nothing here claims exactly-once execution.

Rules are data, evaluators are code



document_extractions + tender + organisation are loaded once into a snapshot the evaluators read

A REAL RULE · migration 002

code	**SBU.NOT_EXPIRED**
severity	**blocker**
document	**SBU**
evaluator	**not_expired_at_deadline**
params	**{ "grace_days": 0 }**

Compares expiry with the submission deadline, not with today.

What the evaluator returns

fail margin < 0 days, with evidence expiresAt, deadline, marginDays

warning valid, but ≤ 30 days after the deadline

pass comfortably valid

inconclusive not processed, no expiry read, or no deadline

Verdict any blocker/critical fail → non_compliant · other fails or warnings → compliant_with_warnings · else compliant

Findings snapshot rule code + severity. Cited rows cannot be deleted (FK RESTRICT), but updates are not blocked: immutability is a convention.

From extracted facts to evidence-backed findings

LIVE RUN · 2026-11-01

non_compliant

score 48.57 · trigger document_ingested

35 findings	**17** pass
18 fail	**0** warning

4 sample PDFs uploaded to the seeded tender (deadline 2026-12-16, HPS Rp 15 bn, requires SBU BG009). 14 fails are mandatory slots not yet filled.

Rule	Severity	Status	Evidence: actual vs expected
SBU.NOT_EXPIRED	blocker	fail	expiresAt 2024-01-10 · deadline 2026-12-16 · marginDays −1072
BOND.AMOUNT_SUFFICIENT	blocker	fail	amount Rp 90,000,000 · requiredMinimum Rp 150,000,000 (1% of HPS)
KSWP.FRESH	critical	fail	ageDays 345 at deadline · maxAgeDays 30
DOC.MANDATORY_PRESENT	blocker	fail ×14	e.g. AKTA_PENDIRIAN has not been uploaded
IDENTITY.NAME_CONSISTENT	major	fail	bid bond: actual "PT Asuransi Nusantara" · expected "PT Karya Bali Mandiri"
SBU.CLASSIFICATION_MATCH	blocker	pass	held [BG009] · required [BG009]
BOND.NOT_EXPIRED	blocker	pass	coverDays 73 · minDays 60

Response shape: GET /v1/compliance-audits/:id → { verdict, score, totals, findings[{ rule, severity, status, message, evidence }] }, worst first. Note: totals.rules counts findings, not rules.

Ingestion-triggered audits: coalesced, one run in flight

`tender_audit_state.pending`

an audit is owed (durable intent)

`tender_audit_state.scheduled_job_id`

the run carrying it (held until it ends)

1 Coalesce

Requests while a run is waiting or delayed only set pending; the debounce (3 s default) collapses a burst onto one run.

2 Guard the active run

The running job keeps scheduled_job_id, so a request mid-audit schedules nothing new.

3 One follow-up

finishRun frees the slot and returns pending in one statement; if set, one follow-up is scheduled after the run ends.

4 Fail safely

Only on the final attempt: pending is restored and the slot freed; the reconciler schedules a fresh run.

Limits. Manual POST …/compliance-audits runs synchronously, outside this coordination. A known race (slot claimed in Postgres, not yet in Redis) can schedule a redundant second run; the fresh test run hit it intermittently. Redundant, never lost.

How an evaluator can explore the system

POST	/v1/tenders	201 · opens mandatory slots
GET	/v1/tenders/:id	tender with its slots
POST	/v1/tenders/:id/documents	202 · multipart, Idempotency-Key
GET	/v1/documents/versions/:id/status	lifecycle + extraction
POST	/v1/tenders/:id/compliance-audits	201 · synchronous audit
GET	/v1/compliance-audits/:auditId	findings, worst first
GET	/v1/document-types	reference data · no header
GET	/healthz · /readyz · /metrics/queue · /docs	probes · Swagger

Every /v1 route except document-types and the demo directory requires X-Organization-Id: <uuid>. It is a simulated tenant context, not authentication. Errors are RFC 7807 problem+json with a code.

DEMO ONLY · OFF BY DEFAULT

GET /v1/demo/organizations

- Read-only; returns id, slug, name, createdAt only
- ?limit=1–100 (50) &offset=0 → pagination {total, limit, offset}
- 404 unless DEMO_MODE=true and NODE_ENV is development, test or demo
- App refuses to start with production + DEMO_MODE=true

```
curl …/v1/demo/organizations
# copy data[0].id, then
curl …/v1/tenders \
  -H "X-Organization-Id: <id>"
```

Fresh run against real PostgreSQL 16 and Redis

210
tests in 22 suites

104
unit · 8 suites

106
integration · 14 suites

209
passed

1
failed (intermittent)

The one failure

audit-scheduling: 'coalesces several requests' saw 2 jobs instead of 1. Alone, 1 failure in 6 reruns: the documented redundant-audit race.

Also passed

npm run typecheck · tsc --noEmit · npm run build · migrations apply cleanly to an empty database.

Mutation evidence

Historical (docs/09): each regression test failed against a deliberate break of its fix. Not re-run here.

Not covered

S3 / MinIO storage path, Docker runtime, production-scale load.

Stated up front, with a path forward

Not built or not guaranteed today

- No authentication: X-Organization-Id is trusted
- No RLS, no composite tenant foreign keys
- No expiry/deadline scheduler: 'scheduled' and 'deadline_approaching' triggers are never produced
- Pagination only on the demo directory; tenders (100) and audit history (20) are capped
- No malware scanning, no rate limiting, no OCR (heuristic text extraction)
- Ruleset and audit_trail immutability are conventions
- A published ingestion job lost in Redis needs a client retry
- Occasional redundant audit under concurrent first requests

Next steps (none implemented)

1. JWT behind the existing guard seam, then RLS and composite tenant keys
2. Repeatable expiry / deadline re-audits
3. Pagination, rate limiting; malware scan on pending → queued
4. Reconcile published-but-lost ingestion jobs
5. Wire presigned uploads; cover S3 in CI
6. AI-assisted extraction, advisory only

Five decisions shape the system

ADR 0001	**SQL-first schema**	Forward-only, checksummed migrations; TypeORM only maps.
ADR 0002	**Asynchronous ingestion**	Commit before publish; 202 and poll for status.
ADR 0003	**Rules as data**	Versioned rows over a closed evaluator registry.
ADR 0004	**Hybrid extraction storage**	Typed columns for queried fields, JSONB for the rest.
ADR 0005	**Audit on its own queue**	Durable intent; audit failure never fails ingestion.

Questions welcome · appendix A1–A7 follows

TECHNICAL APPENDIX

Details behind the walkthrough

A1	Table catalog · tenancy, tenders, documents, processing	slide 19
A2	Table catalog · reliability, compliance, coordination	slide 20
A3	ERD · tender documents and processing	slide 21
A4	ERD · compliance audit	slide 22
A5	Detailed ingestion sequence	slide 23
A6	Failure recovery detail	slide 24
A7	API demo: reproducible calls	slide 25

Tenancy, tenders, documents and processing

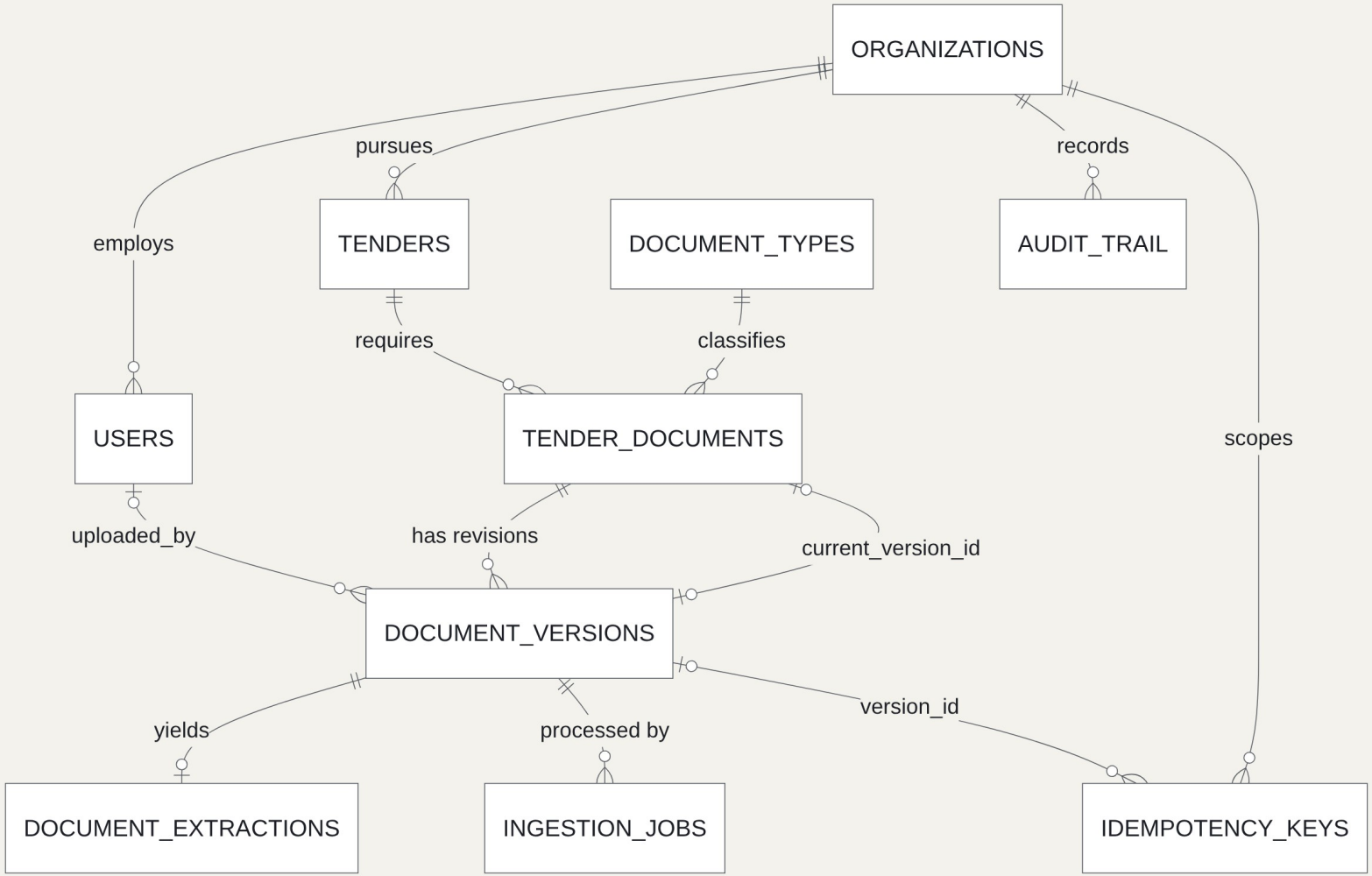
Table	Responsibility	Relationships (database-enforced FKs)	Key constraints
organizations	Tenant root: one contractor company.	Parent of every tenant-owned table (FK, ON DELETE CASCADE).	UNIQUE slug; NPWP format check
users	Members of an organisation; the uploader reference.	organization_id → organizations; referenced by document_versions.uploaded_by (SET NULL).	UNIQUE lower(email); not used for auth yet
tenders	A bid being prepared: code, buyer, deadline, required SBU codes, estimated value.	organization_id → organizations.	UNIQUE (organization_id, tender_code)
document_types	Global reference data: 20 types with MIME and size limits, expiry, mandatory default.	Referenced by tender_documents and compliance_rules (RESTRICT).	PK code; allows_multiple (migration 004)
tender_documents	Slot: one document requirement on a tender.	tender_id → tenders; document_type_code → document_types; current_version_id → document_versions (SET NULL, deferrable).	No UNIQUE (tender, type): multi-instance types
document_versions	One uploaded file; carries the lifecycle.	document_id → tender_documents (CASCADE); uploaded_by → users (SET NULL).	UNIQUE (document_id, version_no); UNIQUE (document_id, checksum)
document_extractions	Fields read from one version: typed columns + extracted_data JSONB + raw diagnostics.	version_id → document_versions (CASCADE).	UNIQUE version_id; confidence 0–1
ingestion_jobs	Ledger and outbox for queue work; published_at = broker accepted it.	version_id → document_versions (CASCADE).	UNIQUE (queue_name, job_key); partial index on unpublished

Reliability, compliance and coordination

Table	Responsibility	Relationships (database-enforced FKs)	Key constraints
idempotency_keys	Claim on a request: fingerprint, stored response, owner_token, linked version; 24 h.	PK (organization_id, key); version_id → document_versions (SET NULL, migration 006).	owner_token per acquisition (migration 004)
compliance_rulesets	Versioned regulation; global, no organization_id.	Parent of compliance_rules (CASCADE); cited by compliance_audits (RESTRICT).	UNIQUE (code, version); one active per code
compliance_rules	One rule: code, severity, evaluator key, JSONB params.	ruleset_id → rulesets (CASCADE); document_type_code → document_types (NULL = tender-level).	UNIQUE (ruleset_id, code)
compliance_audits	One evaluation run: trigger, status, verdict, score, counts.	tender_id → tenders (CASCADE); ruleset_id → rulesets (RESTRICT).	CHECK verdict set ⇔ status completed
compliance_findings	Per-rule result with message, evidence, rule code and severity snapshots.	audit_id → audits (CASCADE); rule_id → rules (RESTRICT); document_id, version_id (SET NULL).	Snapshots are columns, not constraints
tender_audit_state	Durable per-tender audit intent and the job carrying it.	tender_id PK → tenders (CASCADE); organization_id → organizations.	Partial index WHERE pending (migration 004)
audit_trail	Append-only (by convention) record of who changed what.	organization_id → organizations only; entity_id and actor_id are not FKs.	No trigger or grant prevents UPDATE

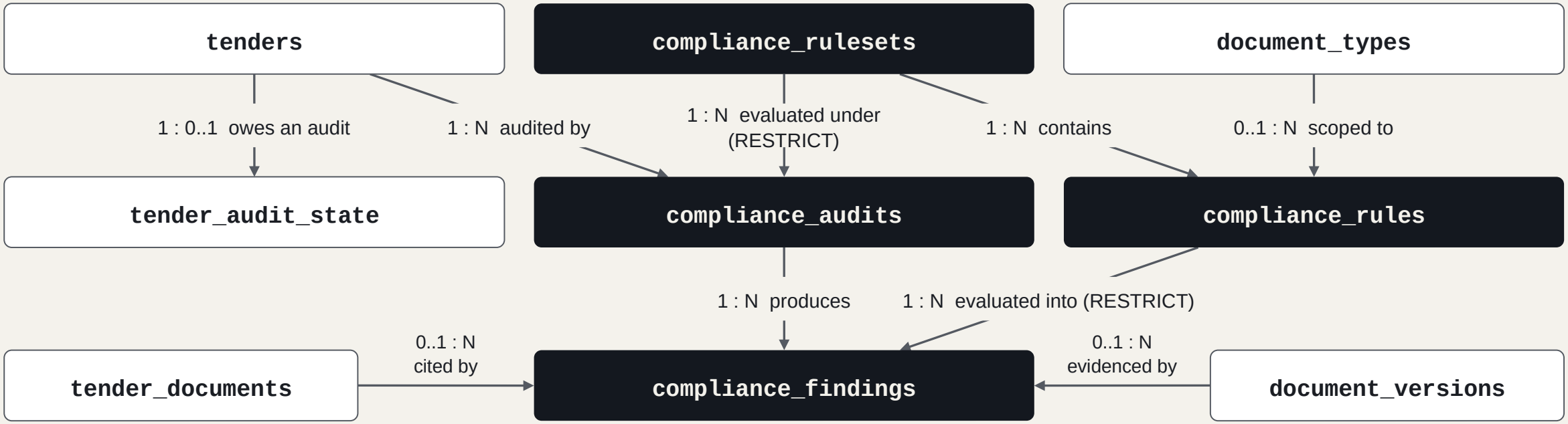
Not shown: schema_migrations (migrator bookkeeping). Tenant consistency between parent and child organization_id is application-enforced.

Tender documents and processing



- Crow's foot = many; circle = optional; bar = exactly one
- current_version_id is a deferrable FK (circular with document_versions)
- Also enforced, not drawn: organization_id FKs on tender_documents, document_versions, ingestion_jobs
- audit_trail.entity_id is polymorphic: no FK

Compliance audit



Arrows point from parent (referenced) to child (referencing) table. Rendered from the docs/ERD.md relationship map and checked against pg_constraint.

Global

Rulesets and rules carry no organization_id: every tenant is audited against the same published regulation.

Delete-protected

Audits → rulesets and findings → rules are ON DELETE RESTRICT. Updates are not blocked.

Intent

tender_audit_state is 1:0..1 with tenders (PK = tender_id); compliance_audits also has an organization_id FK, not drawn.

A5 · INGESTION SEQUENCE

One upload, step by step, with its failure windows

#	Actor	Action	Writes	Window
1	Client	POST /v1/tenders/:id/documents (multipart) + optional Idempotency-Key	–	
2	API	Load tender (404 / 422 deadline) · 8 validation checks · SHA-256 · request fingerprint	–	
3	Postgres	Claim key: new claim, replay stored response, 409, or take over an expired lease	idempotency_keys	C
4	Postgres	Tx: advisory lock (tender, type) · resolve slot · next version_no · insert version pending · link claim · audit_trail	document_versions	A
5	Storage	Put object at org/<org>/<2 hex>/<sha256><ext> unless a complete object of that size exists	object	A
6	Postgres	Tx: UPDATE … SET queued WHERE lifecycle = 'pending' · advance current_version_id (forward only) · insert ingestion_jobs	ingestion_jobs	B
7	Queue	Add job ingest-<versionId> (deterministic id), then set published_at; store response on the claim	published_at	B, E
8	Client	202 Accepted · Location: statusUrl	–	
9	Worker	processing → extract text and fields (heuristic, text-layer PDF)	lifecycle	D
10	Worker	Tx: upsert extraction · processed · job completed · audit_trail (or failed / quarantined)	document_extractions	D
11	Worker	If processed: requestAudit → tender_audit_state.pending (debounced)	tender_audit_state	F
12	Client	GET /v1/documents/versions/:id/status → lifecycle, failure reason, extraction	–	

A6 · FAILURE RECOVERY DETAIL

Who detects each failure, and how it heals

	Failure	Persisted state	Detected by	Recovery	Mode	Evidence
A	**API stops after step 4 or 5**	`version pending; claim.version_id`	Next same-key or same-bytes request	completePendingPromotion: restore object, promote, publish	Client	`promotion-recovery`
B	**Commit ok, publish fails**	`published_at IS NULL`	IngestionReconciler.reconcile after grace	Republish deterministic id (no-op if live)	Auto	`recovery › queue publication`
C	**Request dies holding its key**	`claim, owner_token, version_id`	Same fingerprint after the claim lease	Take over; recover original version; other fingerprint → 409	Client	`idempotent-takeover`
D	**Extraction throws**	`ingestion_jobs failed → dead_letter`	BullMQ retry	Exponential backoff; no text → quarantined (UnrecoverableError)	Auto	`worker-pipeline`
E	**Redis loses a published ingestion job**	`version queued; published_at set`	Not the reconciler; a client retry	republishIfUnpublished: no live job → republish	Client	`code reading only`
F	**Audit run fails**	`compliance_audits failed`	onFailed, final attempt only	restorePending; reconcileAudits schedules a fresh id	Auto	`audit-isolation, audit-followup`
G	**Audit requested mid-run / burst**	`tender_audit_state.pending`	finishRun; conditional slot claim	One follow-up after the run; bursts coalesced (rare redundant run)	Auto	`audit-followup, audit-scheduling`

At-least-once delivery with idempotent handlers. Mode "Client" means recovery needs a retry of the same request.

Reproducible calls, captured from a live run

1 · FIND AN ORGANISATION (DEMO_MODE=true)

```
GET /v1/demo/organizations?limit=5
→ 200 {"data":[{"id":"00000000-…-000000000001",
   "slug":"karya-bali-mandiri","name":"PT Karya Bali Mandiri"}],
   "pagination":{"total":1,"limit":5,"offset":0}}
```

2 · UPLOAD (ORG + TENDER FROM npm run seed)

```
curl -X POST $API/v1/tenders/$TENDER/documents \
  -H "X-Organization-Id: $ORG" -H "Idempotency-Key: demo-1" \
  -F documentTypeCode=SBU -F file=@samples/sbu-expired.pdf
→ 202 {"versionNo":1,"lifecycle":"queued",
   "jobId":"ingest-ce53…","statusUrl":"…/ce53…/status"}
```

3 · SAME KEY, SAME REQUEST

```
→ 202, header Idempotent-Replay: true (no second document)

Status without X-Organization-Id → 400 validation_failed

Not in this capture (covered by tests): same key with a
different file → 409 idempotency_key_reused
```

4 · POLL STATUS

```
GET /v1/documents/versions/ce53…/status
→ 200 {"lifecycle":"processed",
   "extraction":{"documentNumber":"0123456789-BG-2019",
     "issuedAt":"2019-01-10","expiresAt":"2024-01-10",
     "classificationCodes":["BG009"],"confidence":1}}
```

5 · AUDIT HISTORY AND REPORT

```
GET /v1/tenders/$TENDER/compliance-audits
→ [{"status":"completed","verdict":"non_compliant",
    "trigger":"document_ingested","score":"48.57"}]
   (4 uploads → 1 coalesced audit)

GET /v1/compliance-audits/:auditId
→ findings worst first: SBU.NOT_EXPIRED fail,
  BOND.AMOUNT_SUFFICIENT fail, … (see slide 12)

Manual: POST /v1/tenders/$TENDER/compliance-audits
→ 201, synchronous (not run in this capture)
```